In [ ]:
```
from google.colab import files
files.upload()
from google.colab import files
from IPython.display import clear_output

uploaded = files.upload()
clear_output()
```

In [ ]:
```
import pandas as pd

try:
    df = pd.read_csv('/content/Global_Superstore2.csv', encoding='latin1')
except UnicodeDecodeError:
    df = pd.read_csv('/content/Global_Superstore2.csv', encoding='ISO-8859-1')

df.head()
data = df.copy()
data
```

Out[ ]:

	ï»¿Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name
0	40098	CA-2014-AB10015140-41954	11/11/2014	11/13/2014	First Class	AB-100151402	Aaron Bergman
1	26341	IN-2014-JR162107-41675	2/5/2014	2/7/2014	Second Class	JR-162107	Justin Ritter
2	25330	IN-2014-CR127307-41929	10/17/2014	10/18/2014	First Class	CR-127307	Craig Reiter
3	13524	ES-2014-KM1637548-41667	1/28/2014	1/30/2014	First Class	KM-1637548	Katherine Murray
4	47221	SG-2014-RH9495111-41948	11/5/2014	11/6/2014	Same Day	RH-9495111	Rick Hansen
...	...	...	...	...	...	...	...
995	24278	IN-2015-JF1541527-42095	4/1/2015	4/5/2015	Standard Class	JF-1541527	Jennifer Ferguson
996	36249	CA-2015-AS10225140-42347	12/9/2015	12/13/2015	Second Class	AS-102251404	Alan Schoenberger
997	30517	IN-2014-CK1259592-41929	10/17/2014	10/19/2014	Second Class	CK-1259592	Clytie Kelty
998	1408	MX-2014-SS2014018-41658	1/19/2014	1/26/2014	Standard Class	SS-2014018	Saphhira Shifley
999	20944	ID-2015-RP19855130-42348	12/10/2015	12/14/2015	Standard Class	RP-19855130	Roy Phan

1000 rows × 24 columns

In [ ]:

```
top_countries = data.groupby('Country')['Sales'].sum().sort_values(ascending=Fal
top_countries
```

Out[ ]:

Country	Sales
Australia	190368.52
China	160494.10
India	153805.65
France	135114.76
Germany	91884.49

dtype: float64

In [ ]:
```
# what are the top 5 cities with the highest total profit
top_cites = data.groupby('City')['Profit'].sum().sort_values(ascending=False).he
top_cites
```

Out[ ]:

City	Profit
Sydney	6694.32
Detroit	4946.37
Prato	4041.18
Stockton-on-Tees	3769.11
Madrid	3552.45

dtype: float64

In [ ]:
```
#What are the top 5 product categories with the highest total sales?
top_categories = data.groupby('Category')['Sales'].sum().sort_values(ascending=F
top_categories
```

Out[ ]:

Category	Sales
Technology	756983.02
Furniture	601492.31
Office Supplies	352496.14

dtype: float64

In [ ]:
```
#Find the top 5 most profitable products and their total profit
top_products = data.groupby('Product Name')['Profit'].sum().sort_values(ascendin
top_products
```

Out[ ]:

Product Name	Profit
Motorola Smart Phone, Full Size	13088.28
Hoover Stove, Red	10288.61
Cisco Smart Phone, Full Size	9662.99
Cisco Smart Phone, with Caller ID	6607.40
Nokia Smart Phone, Full Size	5765.12

dtype: float64

In [ ]:
```
data.head(1)
```

Out[ ]:

	ï»¿Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segmer
0	40098	CA-2014-AB10015140-41954	11/11/2014	11/13/2014	First Class	AB-100151402	Aaron Bergman	Consum

1 rows × 24 columns

In [ ]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 24 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   ï»¿Row ID       1000 non-null   int64
 1   Order ID        1000 non-null   object
 2   Order Date      1000 non-null   object
 3   Ship Date       1000 non-null   object
 4   Ship Mode       1000 non-null   object
 5   Customer ID     1000 non-null   object
 6   Customer Name   1000 non-null   object
 7   Segment         1000 non-null   object
 8   Postal Code     194 non-null    float64
 9   City            1000 non-null   object
 10  State           1000 non-null   object
 11  Country         1000 non-null   object
 12  Region          1000 non-null   object
 13  Market          1000 non-null   object
 14  Product ID      1000 non-null   object
 15  Category        1000 non-null   object
 16  Sub-Category    1000 non-null   object
 17  Product Name    1000 non-null   object
 18  Sales           1000 non-null   float64
 19  Quantity        1000 non-null   int64
 20  Discount        1000 non-null   float64
 21  Profit          1000 non-null   float64
 22  Shipping Cost   1000 non-null   float64
 23  Order Priority  1000 non-null   object
dtypes: float64(5), int64(2), object(17)
memory usage: 187.6+ KB
```

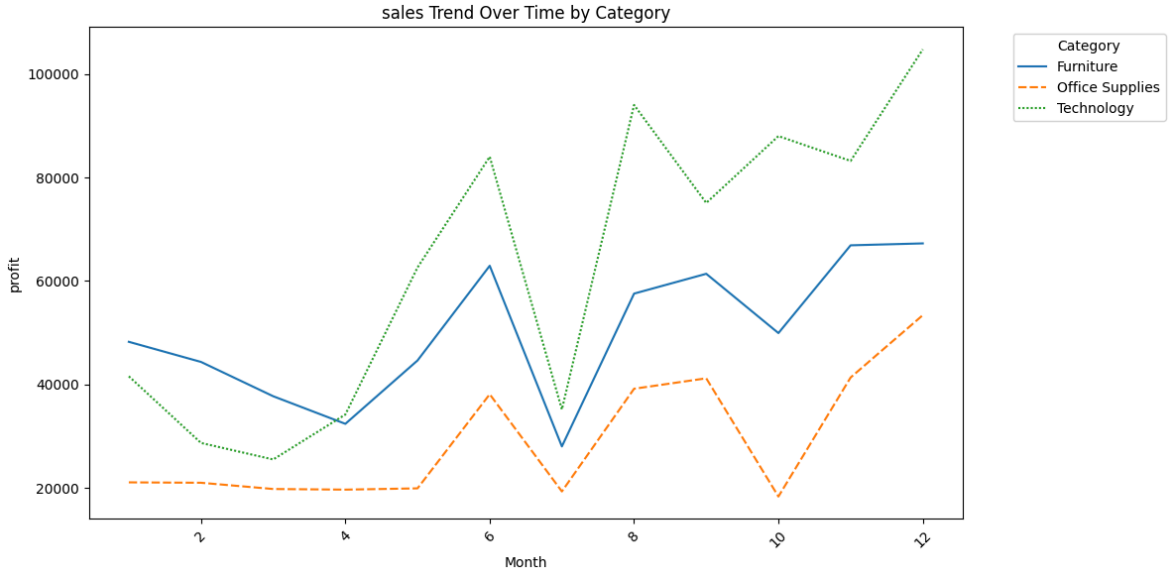
In [ ]:
```
data['Order Date'] = pd.to_datetime(data['Order Date'])
data['month'] = data['Order Date'].dt.month
```

In [ ]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

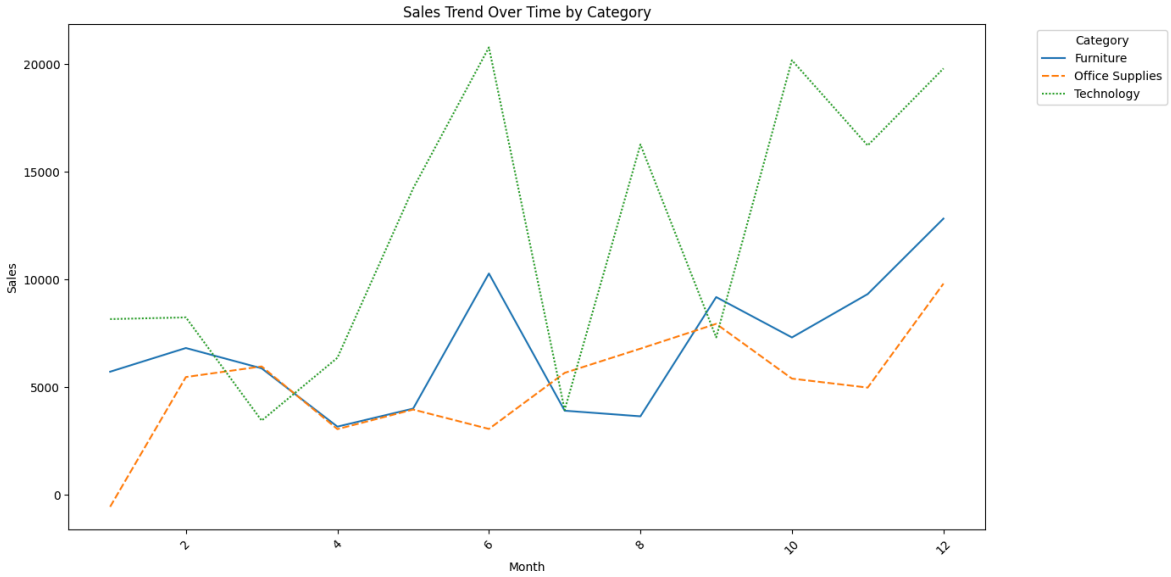
In [ ]:
```
#How do sales and profit trends differ by product category    over time?
sales_trends = data.groupby(['month', 'Category'])['Sales'].sum().reset_index()
profit_trends = data.groupby(['month', 'Category'])['Profit'].sum().reset_index(
```

In [ ]:
```
# Pivot tables for better plotting
sales_pivot = sales_trends.pivot(index='month', columns='Category', values='Sale
profit_pivot = profit_trends.pivot(index='month', columns='Category', values='Pr
```

In [ ]:
```
# Plot Sales Trend
plt.figure(figsize=(12, 6))
sns.lineplot(data=sales_pivot)
plt.title('sales Trend Over Time by Category')
plt.xlabel('Month')
plt.ylabel('profit')
plt.legend(title='Category',bbox_to_anchor=(1.05, 1), loc='upper left')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



In [ ]:
```
plt.figure(figsize=(14, 7))
sns.lineplot(data=profit_pivot)
plt.title('Sales Trend Over Time by Category')
plt.xlabel('Month')
plt.ylabel('Sales')
plt.legend(title='Category', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



In [ ]:
```
data.head(2)
```

Out[ ]:

	ï»¿Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment
0	40098	CA-2014-AB10015140-41954	2014-11-11	11/13/2014	First Class	AB-100151402	Aaron Bergman	Consumer
1	26341	IN-2014-JR162107-41675	2014-02-05	2/7/2014	Second Class	JR-162107	Justin Ritter	Corporate

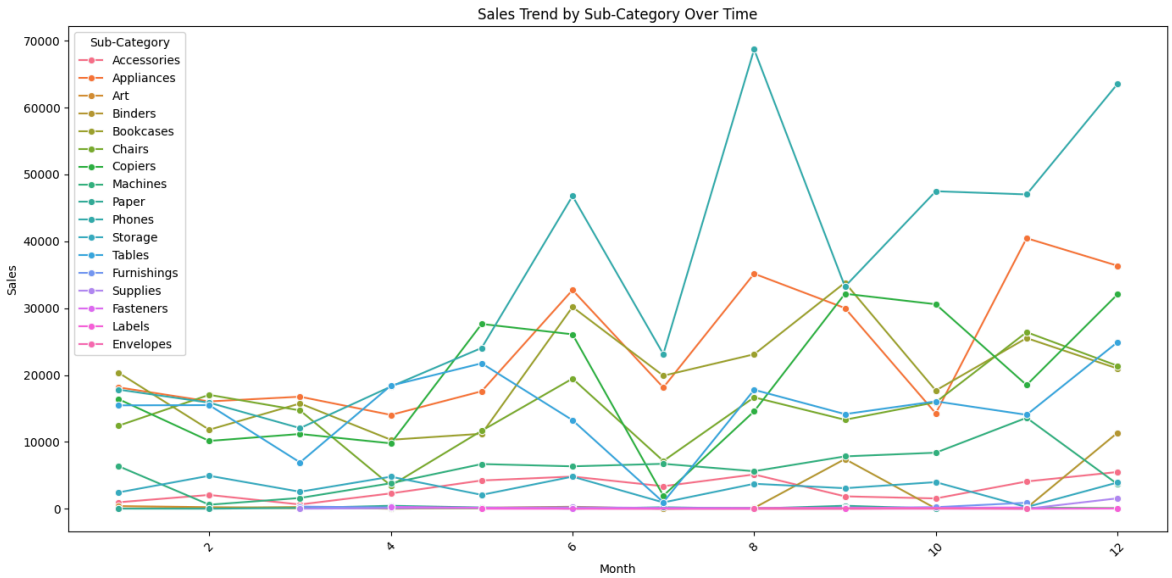
2 rows × 25 columns

In [ ]:
```
#How do sales and profit trends differ by product sub-category over time?
sub_sales_trend = data.groupby(['month' , 'Sub-Category'])['Sales'].sum().reset_
sub_profit_trend = data.groupby(['month' , 'Sub-Category'])['Profit'].sum().rese
```

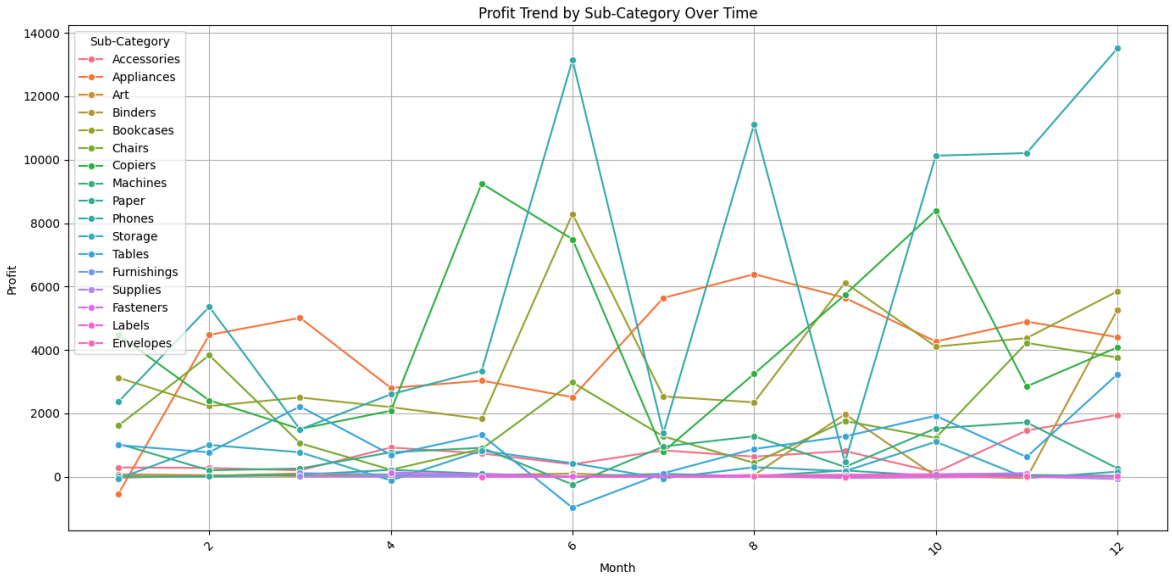
In [ ]:
```
#pivoting
sub_sales_pivot = sub_sales_trend.pivot(index='month' , columns='Sub-Category' ,
sub_profit_pivot = sub_profit_trend.pivot(index='month' , columns='Sub-Category'
```

In [ ]:
```
#plotting1
sub_sales_trend.reset_index(inplace=True)
sub_profit_trend.reset_index(inplace=True)
```

In [ ]:
```
# Plot Sales Trend by Sub-Category
plt.figure(figsize=(14, 7))
sns.lineplot(data=sub_sales_trend, x='month' , y='Sales' ,hue='Sub-Category',mar
plt.title('Sales Trend by Sub-Category Over Time')
plt.xlabel('Month')
plt.ylabel('Sales')
plt.legend(title='Sub-Category')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



In [ ]:
```
# Plot Profit Trend by Sub-Category
plt.figure(figsize=(14, 7))
sns.lineplot(data=sub_profit_trend, x='month', y='Profit', hue='Sub-Category', m
plt.title('Profit Trend by Sub-Category Over Time')
plt.xlabel('Month')
plt.ylabel('Profit')
plt.legend(title='Sub-Category')
plt.xticks(rotation=45)
plt.tight_layout()
plt.grid(True)
plt.show()
```



In [ ]:
```
data.head(1)
```

Out[ ]:

	ï»¿Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	
0	40098	CA-2014-AB10015140-41954	2014-11-11	11/13/2014	First Class	AB-100151402	Aaron Bergman	Consumer	7

1 rows × 25 columns

In [ ]:
```
#Understanding co relation between discount levels and Profit Margins
data['Profit Margin'] = data['Profit'] / (data['Sales'])
# Calculate correlation between discount and profit margin
correlation = data[['Discount', 'Profit Margin']].corr().iloc[0, 1]
print(f"Correlation between Discount and Profit Margin: {correlation:.2f}")
```

```
Correlation between Discount and Profit Margin: -0.78
```

In [ ]:
```
data['Country'].nunique()
```

Out[ ]: 93

In [ ]:
```
data.head(1)
```

Out[ ]:

	ï»¿Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	
0	40098	CA-2014-AB10015140-41954	2014-11-11	11/13/2014	First Class	AB-100151402	Aaron Bergman	Consumer	7

1 rows × 26 columns

In [ ]:
```
City_Sales = data.groupby('City')['Sales'].agg(['sum', 'mean']).reset_index()
City_Sales.columns = ['City', 'Total Sales', 'Average Sales']
```

In [ ]:
```
# Total and average sales for each state
State_sales = data.groupby('State')['Sales'].agg(['sum', 'mean']).reset_index()
State_sales.columns = ['State', 'Total Sales', 'Average Sales']
```

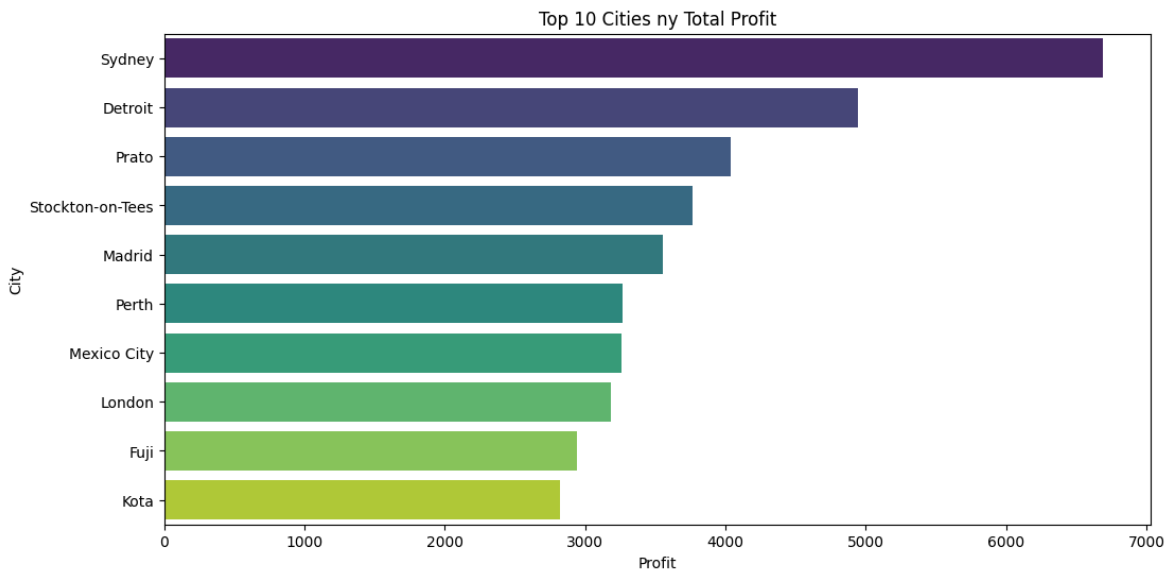
In [ ]:
```
# Total and average sales for each country
Country_sales = data.groupby('Country')['Sales'].agg(['sum', 'mean']).reset_inde
Country_sales.columns = ['Country', 'Total Sales', 'Average Sales']
```

In [ ]:
```
# Total and average sales for each region
Region_sales = data.groupby('Region')['Sales'].agg(['sum', 'mean']).reset_index(
Region_sales.columns = ['Region', 'Total Sales', 'Average Sales']
```

In [ ]:
```
#Objective: Understand the total profit and profit margins across different loca
  #city
  # Total profit for each city
city_profit = data.groupby('City')['Profit'].sum().reset_index().sort_values(by=
```

In [ ]:
```
# Visualize the top and bottom cities by profit
plt.figure(figsize=(12, 6))
sns.barplot(x='Profit', y='City', data=city_profit.head(10), palette='viridis')
plt.title('Top 10 Cities ny Total Profit')
plt.show()
```

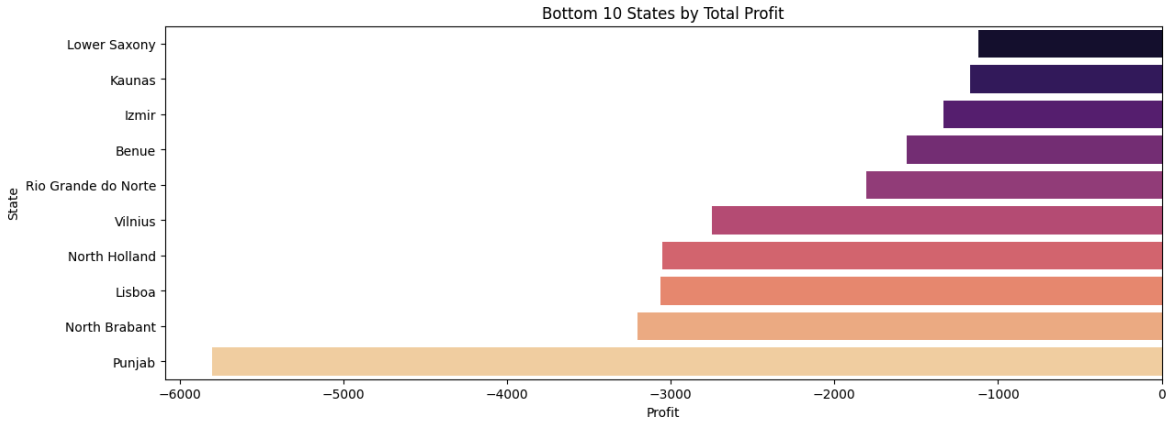
```
/tmp/ipykernel_9623/4020265858.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Profit', y='City', data=city_profit.head(10), palette='viridis')
```



In [ ]:
```
state_profit = data.groupby('State')['Profit'].sum().reset_index().sort_values(b
```

In [ ]:
```
state_profit = data.groupby('State')['Profit'].sum().reset_index().sort_values(b
plt.figure(figsize=(14, 5))
sns.barplot(x='Profit', y='State', data=state_profit.tail(10), palette='magma')
plt.title('Bottom 10 States by Total Profit')
plt.show()
```

```
/tmp/ipykernel_9623/3474279682.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Profit', y='State', data=state_profit.tail(10), palette='magma')
```

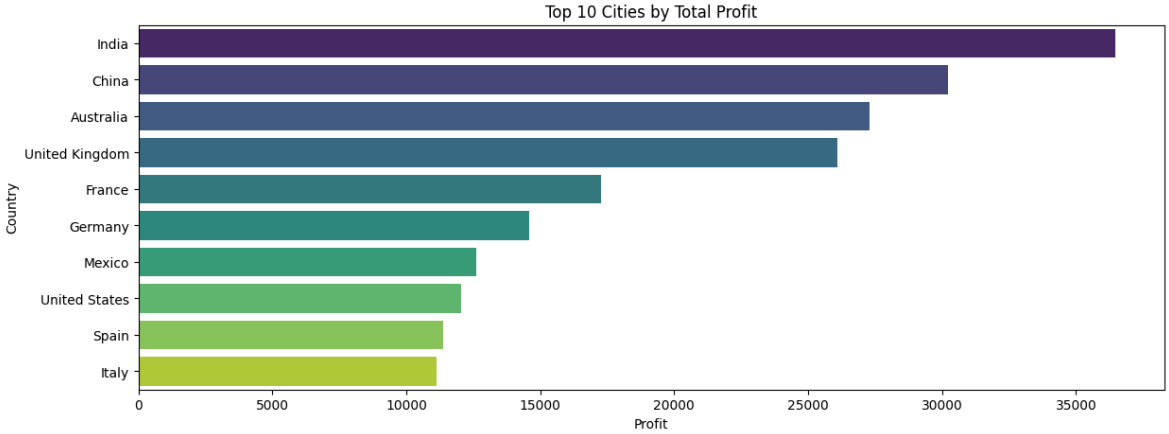


In [ ]:
```
#country
# Total profit for each Country
Country_profit = data.groupby('Country')['Profit'].sum().reset_index().sort_valu
```

In [ ]:
```
# Visualize the top and bottom cities by profit
plt.figure(figsize=(14, 5))
sns.barplot(x='Profit', y='Country', data=Country_profit.head(10), palette='viri
plt.title('Top 10 Cities by Total Profit')
plt.show()

plt.figure(figsize=(14, 5))
```

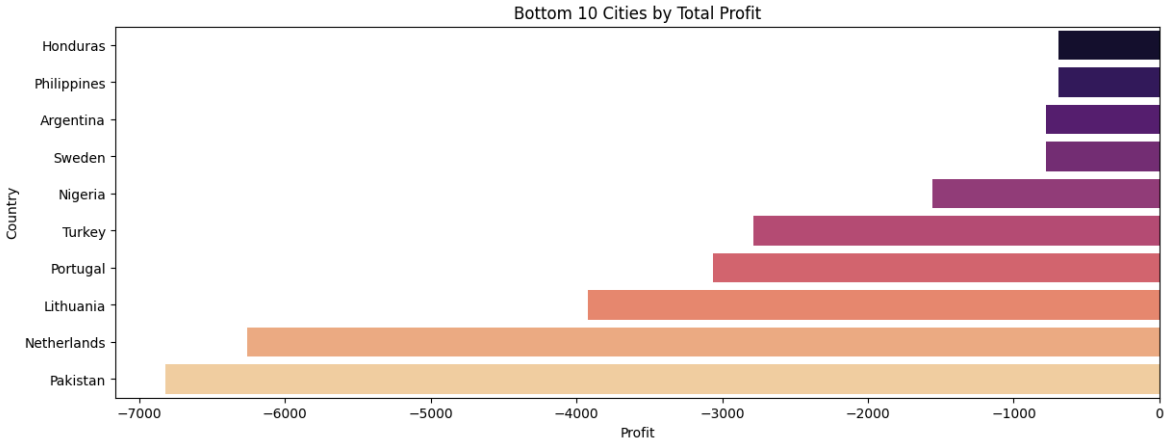
```
sns.barplot(x='Profit', y='Country', data=Country_profit.tail(10), palette='magm
plt.title('Bottom 10 Cities by Total Profit')
```

```
/tmp/ipykernel_9623/3560755222.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Profit', y='Country', data=Country_profit.head(10), palette='vir
idis')
```




```
/tmp/ipykernel_9623/3560755222.py:8: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Profit', y='Country', data=Country_profit.tail(10), palette='mag
ma')
```

Out[ ]: Text(0.5, 1.0, 'Bottom 10 Cities by Total Profit')



In [ ]:

```
city_profit
```

Out[ ]:

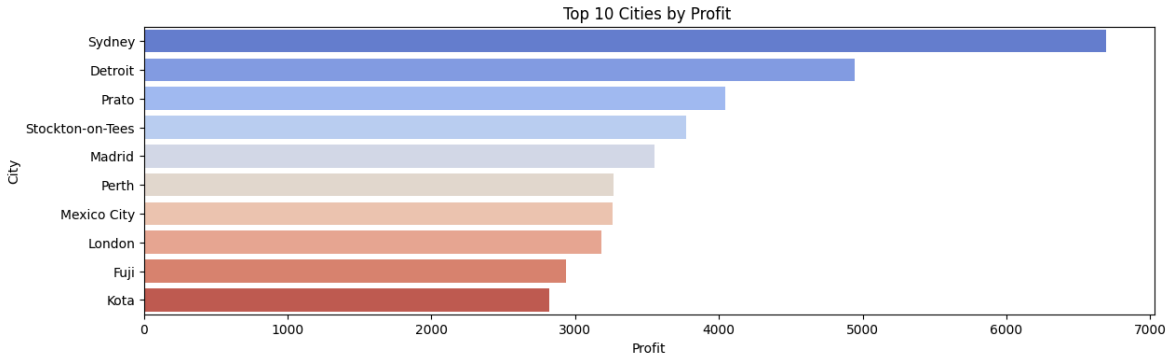
	City	Profit
502	Sydney	6694.32
141	Detroit	4946.37
420	Prato	4041.18
493	Stockton-on-Tees	3769.11
314	Madrid	3552.45
...	...	...
63	Bergen op Zoom	-2211.17
557	Vilnius	-2750.28
11	Amsterdam	-3052.94
299	Lisbon	-3059.82
280	Lahore	-5799.36

592 rows × 2 columns

In [ ]:
```
#Profit Margin based on city and region

# ProfMarginit margin for each city
data['Profit ']= (data['Profit'] / data['Sales'])*100
city_profit_margin = data.groupby('City')['Profit'].mean().reset_index().sort_va
```

In [ ]:
```
# Visualize the cities with the highest and lowest profit margins
plt.figure(figsize=(14, 4))
sns.barplot(x='Profit', y='City', data=city_profit.head(10), palette='coolwarm')
plt.title('Top 10 Cities by Profit ')
plt.show()
```

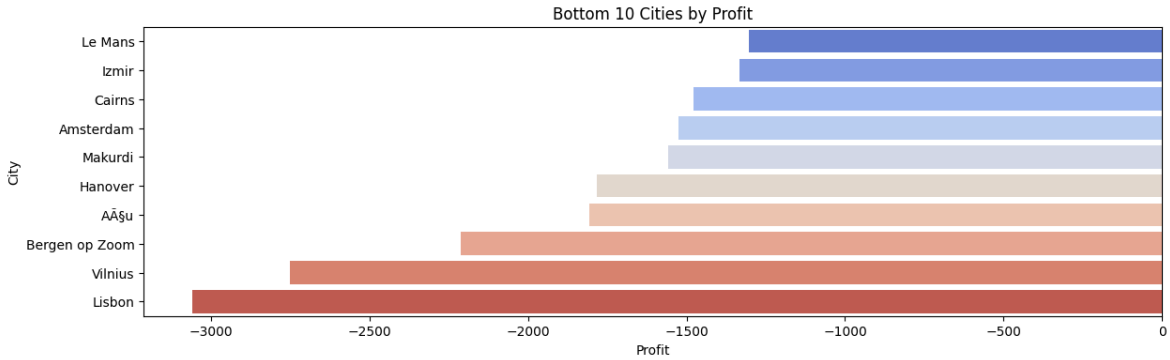
```
/tmp/ipykernel_9623/353401179.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Profit', y='City', data=city_profit.head(10), palette='coolwar
m')
```



In [ ]:
```
plt.figure(figsize=(14, 4))
sns.barplot(x='Profit', y='City', data=city_profit_margin.tail(10), palette='coo
```

```
plt.title('Bottom 10 Cities by Profit')
plt.show()
```

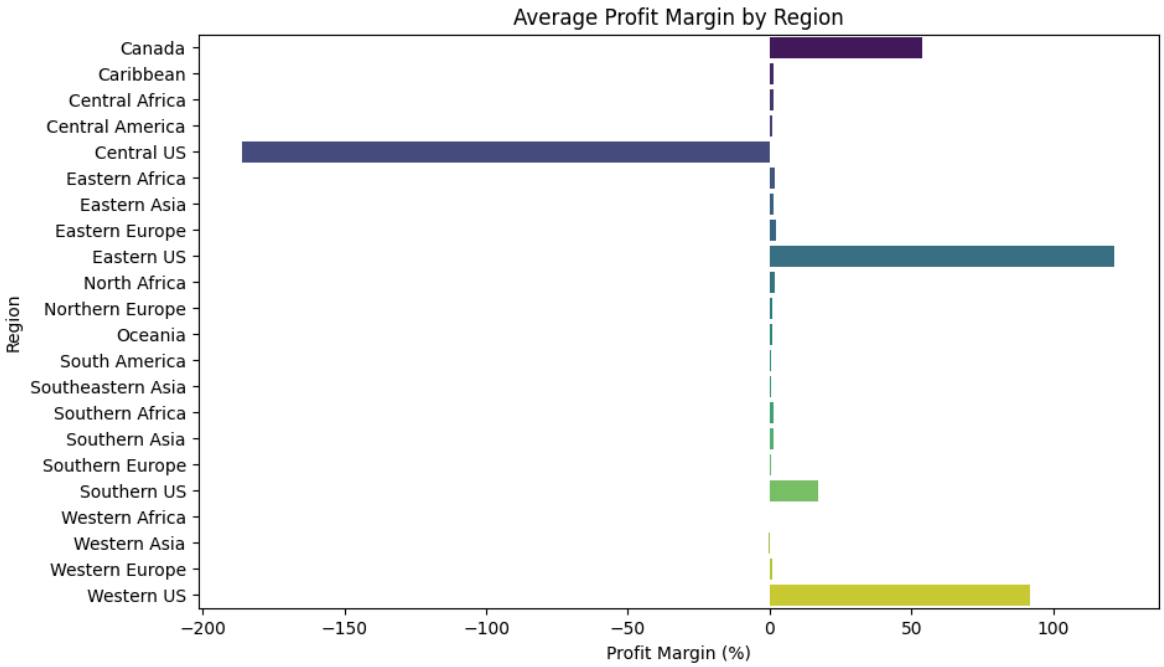
```
/tmp/ipykernel_9623/6070053.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Profit', y='City', data=city_profit_margin.tail(10), palette='co
olwarm')
```



In [ ]:
```
# Calculate profit margin by region

data['Profit'] = (data['Profit'] / data['Sales']) * 100
Profit = data.groupby('Region')['Profit'].mean().reset_index()
```

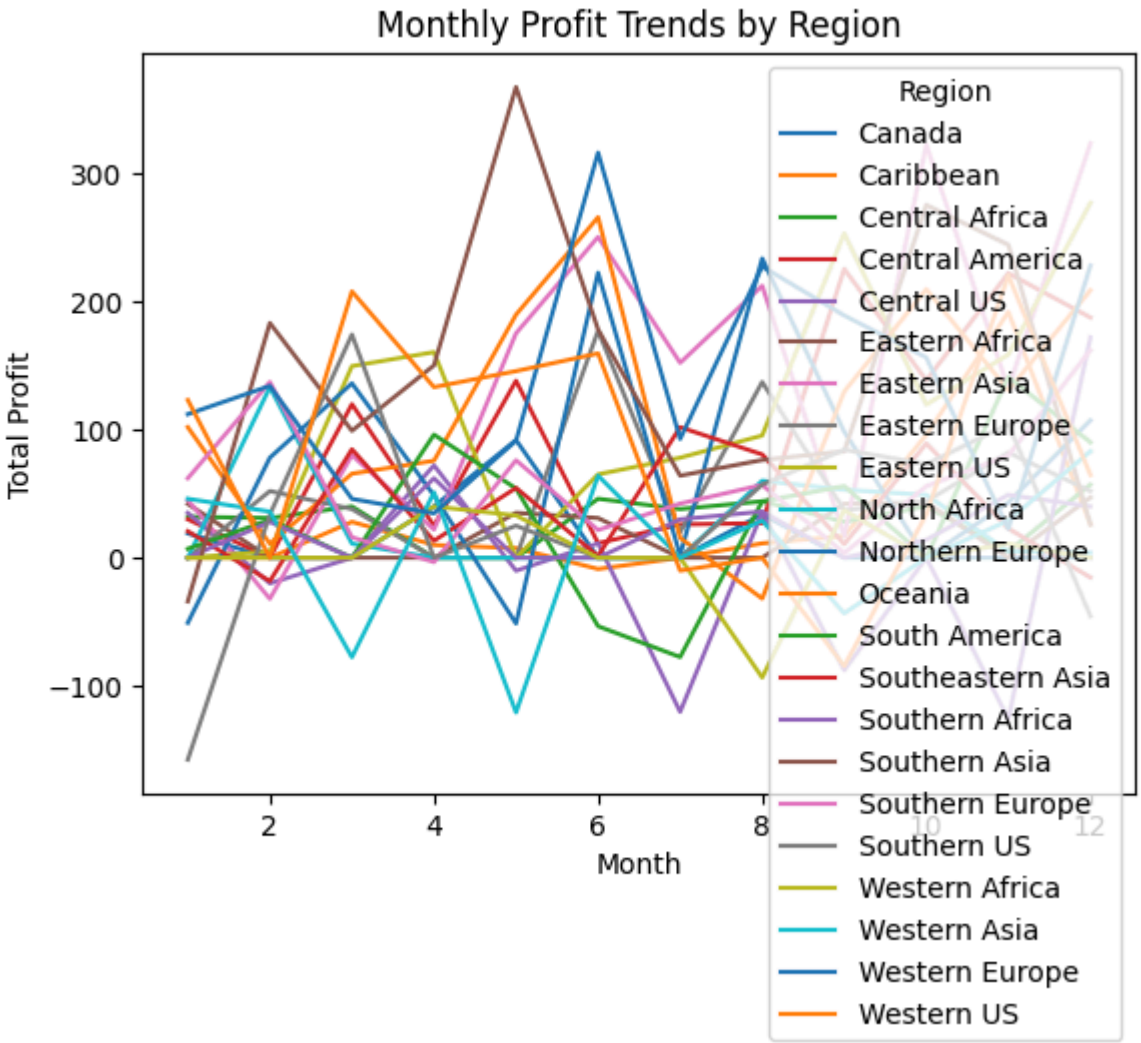
In [ ]:
```
# plotting
plt.figure(figsize=(10, 6))
sns.barplot(x='Profit Margin', y='Region', data=profit_margin_region, palette='v
plt.title('Average Profit Margin by Region')
plt.xlabel('Profit Margin (%)')
plt.ylabel('Region')
plt.show()
```

```
/tmp/ipykernel_9623/2246947785.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Profit Margin', y='Region', data=profit_margin_region, palette
='viridis')
```



In [ ]:
```
#Profitability Over Time by Region
monthly_profit = data.groupby(['month', 'Region'])['Profit'].sum().unstack().fil
# plotting
plt.figure(figsize=(25, 7))
monthly_profit.plot(kind='line')
plt.title('Monthly Profit Trends by Region')
plt.ylabel('Total Profit')
plt.xlabel('Month')
plt.show()
```

```
<Figure size 2500x700 with 0 Axes>
```



In [ ]:

```
monthly_profit['Total Profit'] = monthly_profit.sum(axis=1)

# Sort the DataFrame by 'Total Profit' in descending order
monthly_profit_sorted = monthly_profit.sort_values(by='Total Profit', ascending=

# Display the top 4 rows
monthly_profit_sorted.head(4)
```

Out[ ]:

Region	Canada	Caribbean	Central Africa	Central America	Central US	Eastern Africa	Eastern Asia
month							
12	0.0	26.240987	56.997346	187.282084	171.697403	46.997660	322.932028
11	0.0	191.606185	5.000000	221.803065	-125.718355	0.000000	131.986192
10	0.0	95.838506	39.998158	138.369644	0.000000	42.989105	321.847233
6	0.0	-8.750053	45.999623	11.325716	11.250478	30.980392	250.062900

4 rows × 23 columns